EFREI Paris — Cycle ingénieur ING-APP1-BDML
Algorithmique et théorie des graphes — Projet intégrateur

Novapolis Résilience

Planification des secours et conception d'un réseau urbain résilient

Rapport de projet

Code source : `https://github.com/AjdahimHassan/TP_GRAPHES`

Hassan AJDAHIM (membre 1)
Andreja JOVANOVIC (membre 2)
Angelo CUNHA VELHO MACHADO (membre 3)

Données synthétiques. Novapolis est une ville fictive. Toutes les données utilisées dans ce rapport ont été produites par notre propre générateur, avec des graines fixées. N'importe quel tableau ou figure peut donc être refait à l'identique avec la commande indiquée en annexe A.

Novembre 2026

Synthèse

La ville de Novapolis voulait un outil capable de répondre à trois questions, et surtout un outil dont on puisse refaire les calculs. Pour ça, nous avons représenté la ville par deux graphes construits sur les mêmes 150 sites. Le premier est le réseau routier : il est orienté, parce qu'une rue peut être à sens unique, et chaque route porte un temps de trajet. Le second est le réseau de fibre, qui n'a pas de sens, et où chaque tronçon a un coût de construction. Tous les algorithmes ont été codés par nous en Python, sans bibliothèque de graphes. On a seulement utilisé `networkx` dans les tests, pour vérifier qu'on trouvait les mêmes résultats.

Voici, en résumé, ce que l'outil répond sur le jeu de données réaliste :

Question de la ville	**Ce qu'on a trouvé**
1. Quels quartiers sont mal desservis ?	149 sites sur 150 sont atteignables depuis les casernes. Le quartier `Q-123` est complètement isolé, et 9 quartiers sont à plus de 10 minutes de toute caserne ou de tout hôpital.
2. Quels itinéraires si des routes ferment ?	Quand on ferme les 3 ponts du centre, personne ne devient inaccessible. Par contre 4 interventions sont retardées, la pire de 6,56 minutes (presque deux fois plus long) pour `D-02`. Les trois algorithmes de plus court chemin trouvent les mêmes distances.
3. Quel réseau de fibre, et comment le réparer ?	Le réseau le moins cher coûte 11 130 k€ avec 147 tronçons, que ce soit avec Kruskal ou avec Prim. Si le tronçon `Q-014`–`Q-019` tombe en panne, on le remplace par `Q-014`–`Q-025` (80 k€), ce qui augmente le coût total de seulement 15 k€.

Sommaire

1 Le problème et notre démarche 2
2 Modélisation, données et architecture 2
3 Module 1 — Qui peut être atteint ? (DFS, BFS) 5
4 Module 2 — Itinéraires des secours 5
5 Module 3 — Les circuits absorbants 8
6 Module 4 — Le réseau de fibre 9
7 Module 5 — Résilience et scénarios 10
8 Tests, robustesse et reproductibilité 12
9 Mesures de performance et analyse 12
10 Limites et recommandations 14
11 Démonstration et contributions 14
A Annexe — Commandes pour tout refaire 15

1 Le problème et notre démarche

Novapolis, c'est une ville avec des hôpitaux, des casernes de pompiers, des écoles, des centres de télécommunication et beaucoup de quartiers d'habitation. Après plusieurs incidents, la mairie se pose trois questions. D'abord, quels quartiers sont trop loin des secours ? Ensuite, quand des routes sont bouchées ou fermées, quel chemin doivent prendre les pompiers ? Et enfin, comment construire un réseau de fibre pas trop cher, qui puisse être réparé facilement si un tronçon casse ?

Le sujet le dit clairement : le but n'est pas de faire une jolie carte. Ce qui compte, c'est que les résultats soient justes et qu'on puisse le prouver. On a donc essayé de respecter trois règles pendant tout le projet. La première, c'est que tout doit être vérifiable : chaque algorithme a été testé sur un petit graphe qu'on a calculé à la main, puis comparé à `networkx` sur 50 graphes tirés au hasard. La deuxième, c'est que les scénarios doivent être comparables : on applique chaque perturbation à une copie de la ville, et on mesure toujours les mêmes indicateurs qu'en situation normale. La troisième, c'est la reproductibilité : les données sortent d'un générateur avec une graine fixe, et chaque résultat est accompagné de la commande qui permet de le refaire.

Concrètement, l'outil peut s'utiliser en ligne de commande (`novapolis ...`), par une API, ou par une interface web. Les trois appellent exactement le même code Python. L'interface web ne fait aucun calcul elle-même, elle demande juste les résultats à l'API, donc les chiffres affichés sont forcément les mêmes que dans le terminal. On a d'ailleurs écrit un test pour le vérifier.

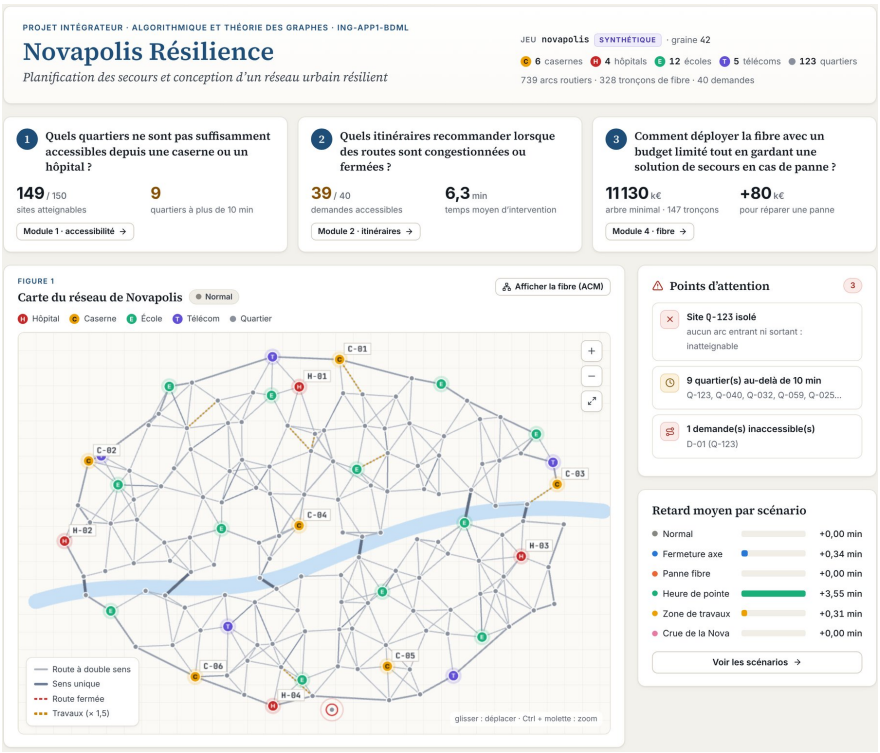


Figure 1 – Page d'accueil de l'interface. On y retrouve les trois questions de la ville, les réponses et la carte du jeu réaliste. Les losanges orange en pointillés montrent des routes en travaux.

2 Modélisation, données et architecture

2.1 Deux graphes pour une même ville

Le choix le plus important du projet a été de séparer la ville en deux graphes. Pour les routes, on utilise un graphe orienté : une flèche de A vers B veut dire qu'on peut rouler de A vers B. Une rue à double sens donne donc deux flèches, une dans chaque sens, alors qu'un sens unique n'en donne qu'une. Le poids de chaque flèche, c'est le temps de trajet en minutes. Pour la fibre, le sens n'a pas

d'importance (un câble relie deux points, c'est tout), donc on utilise un graphe non orienté où le poids est le coût de construction en milliers d'euros.

Il a aussi fallu décider quel temps exactement donner aux algorithmes, parce que les fichiers contiennent plusieurs informations. Le temps utilisé est le temps normal ou le temps de pointe, selon la période du scénario. Une route fermée est tout simplement retirée du graphe. Une route en travaux reste utilisable, mais son temps est multiplié par 1,5. C'est un choix de notre part, le sujet ne donnait pas de valeur. Côté fibre, les tronçons interdits ou en panne ne sont jamais proposés aux algorithmes. Pour savoir quelle caserne envoie les secours, on prend simplement celle qui arrive le plus vite. Plutôt que de lancer un calcul par caserne, on lance un seul Dijkstra qui part de toutes les casernes en même temps, ce qui donne directement la plus proche. Enfin, les temps négatifs n'ont pas de sens dans une vraie ville. Le validateur les refuse donc partout, sauf dans les deux petits jeux de test du module 3, où ils sont autorisés exprès.

2.2 Les fichiers et le validateur

Un jeu de données, c'est un dossier avec les cinq fichiers CSV demandés par le sujet (sites, routes, fibre, demandes, scénarios) et un petit fichier `meta.json`. Ce dernier garde la graine utilisée, les paramètres du générateur, et une empreinte de chaque fichier pour vérifier plus tard qu'il n'a pas changé. On a ajouté deux colonnes facultatives (le type de voie et la capacité), qui servent seulement à l'affichage. Tout le format est décrit dans `data/SCHEMA.md`.

Avant de lancer le moindre calcul, le validateur relit tous les fichiers. Quand il trouve un problème, il donne le fichier, la ligne, la colonne et la valeur qui pose souci, pour qu'on puisse corriger rapidement. Pour le tester, on a volontairement ajouté deux erreurs dans le petit jeu `mini`, et voilà ce qu'il répond :

Gravité	Fichier	Ligne	Colonne	Message
erreur	`sites.csv`	10	`id`	Identifiant dupliqué C1 (déjà défini ligne 2)
erreur	`routes.csv`	16	`v`	Site inexistant ZZZ : arc rejeté

Il repère aussi les fichiers ou colonnes qui manquent, les types ou statuts qu'il ne connaît pas, les valeurs qui ne sont pas des nombres, les temps ou coûts négatifs, et les priorités qui ne valent pas 1, 2 ou 3. Certaines choses ne sont pas vraiment des erreurs, comme une route en double ou un scénario qui ne change rien : dans ce cas il affiche seulement un avertissement.

2.3 Les jeux de données

On a préparé plusieurs jeux de données, chacun avec un rôle précis (tableau 1). Le plus utile pendant le développement a été le jeu `mini` : avec seulement 8 sites, on peut tout calculer à la main et comparer.

Le générateur donne toujours le même résultat : avec la même graine, on obtient exactement les mêmes fichiers, octet par octet, qu'on les génère depuis l'interface ou depuis le terminal. Ça a demandé un peu d'attention (trier les lignes, écrire les nombres toujours avec le même format), mais c'est ce qui permet de refaire nos expériences. Un test vérifie que c'est bien le cas.

2.4 Pourquoi ces structures de données

Pour chaque algorithme, on a choisi la structure qui lui permet d'avoir la complexité vue en cours. La plupart des algorithmes (les parcours, Dijkstra, Prim, Tarjan) utilisent des **listes d'adjacence** : pour chaque site, on garde la liste de ses voisins. C'est ce qui permet de parcourir tout le graphe en un temps proportionnel au nombre de sites plus le nombre de routes, et ça prend peu de mémoire, ce qui tombe bien parce qu'un réseau routier a peu de routes par carrefour. Kruskal et Bellman–Ford, eux, travaillent plutôt sur une simple **liste de toutes les arêtes** : Kruskal doit les trier par coût, et Bellman–Ford les repasse toutes à chaque tour.

Floyd–Warshall est différent, puisqu'il remplit une **matrice** de toutes les distances. L'accès à une case est immédiat, mais la matrice grossit avec le carré du nombre de sites. C'est pour ça qu'on ne l'utilise

Jeu	Sites	Arcs	Tronçons	À quoi il sert
`mini`	8	14	10	Vérification à la main. Les résultats attendus sont dans `data/mini/EXPECTED.md` et les tests les contrôlent.
`negatif_accessible`	6	7	0	Test A du module 3 : le circuit négatif est atteignable.
`negatif_inaccessible`	6	7	0	Test B du module 3 : le circuit négatif n'est pas atteignable.
`novapolis`	150	739	328	Le jeu réaliste (graine 42) : 6 casernes, 4 hôpitaux, 12 écoles, 5 centres télécom, 123 quartiers, une rivière traversée par 6 ponts, un périphérique, 10 % de sens uniques, 40 demandes et 6 scénarios.
`n1000...n5000`	1 000–5 000	2 888–57 061	1 280–25 437	Les 9 gros jeux des benchmarks (3 tailles et 3 densités).

Table 1 – Les jeux de données fournis. Ils sont tous synthétiques, et l'interface l'indique.

que sur des graphes de 400 sites au plus. Pour Dijkstra et Prim, on a besoin de récupérer à chaque fois l'élément le plus petit : on utilise donc un **tas** (le module `heapq` de Python). Le BFS utilise une **file** (`deque`), et Kruskal un **Union–Find** pour savoir rapidement si deux sites sont déjà reliés. Enfin, tous nos parcours sont écrits sans récursion, avec une pile qu'on gère nous-mêmes. Comme ça, Python ne plante pas sur les gros graphes : un des tests parcourt une chaîne de 20 000 sites sans problème.

2.5 Organisation du code

Le code est découpé en plusieurs parties qui ont chacune un seul rôle, comme le demande le sujet (figure 2). Les algorithmes ne savent rien de l'affichage ni du format des fichiers : ils reçoivent un graphe et renvoient un résultat. Entre les deux, une couche `services` propose une fonction par opération, et c'est elle qu'appellent à la fois la ligne de commande et l'API.

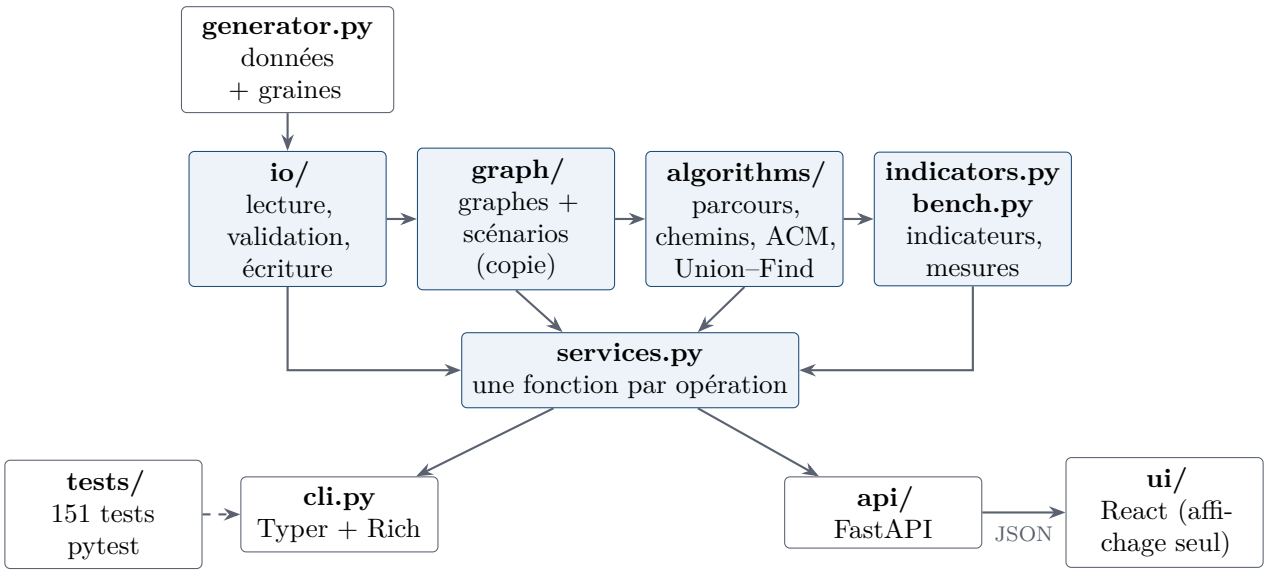


Figure 2 – Organisation du code. Chaque bloc a un rôle, et les algorithmes ne dépendent pas de l'affichage.

Depuis le terminal, on choisit le jeu de données, le scénario et l'opération à lancer. Par exemple, pour calculer l'itinéraire de la demande D-02 avec les trois algorithmes quand les ponts sont fermés :

```
novapolis route -data data/novapolis -demande D-02 -algo all -scenario fermeture_axe
```

3 Module 1 — Qui peut être atteint ? (DFS, BFS)

3.1 Attention aux différents sens de « connecté »

Dans un graphe orienté, dire que la ville est « connectée » peut vouloir dire plusieurs choses, et on s'est rendu compte qu'il fallait bien les séparer. La notion qui compte vraiment pour les secours, c'est l'**accessibilité** : est-ce qu'il existe un chemin, en respectant le sens des rues, entre une caserne et un site ? C'est ce qu'on calcule avec un BFS qui part de toutes les casernes en même temps. La **connexité faible**, elle, ignore le sens des rues : elle sert juste à voir si certains morceaux de la ville sont complètement coupés du reste. Et la **connexité forte** demande qu'on puisse aller de A à B *et* revenir de B à A. On la calcule avec l'algorithme de Tarjan, et elle permet de repérer des endroits où l'on peut entrer sans pouvoir ressortir. Pour la fibre, qui n'a pas de sens, la question ne se pose pas : on compte juste les composantes connexes.

3.2 Vérification à la main sur le petit jeu

Le jeu `mini` (figure 3) a été construit pour contenir tous les cas un peu piégeux : un sens unique (de H1 vers E1), une route fermée (de Q1 vers E1), une route en travaux (de C2 vers Q2, qui passe de 6 à 9 minutes), un site dont on ne peut pas ressortir (E1), et un site isolé (Q3).

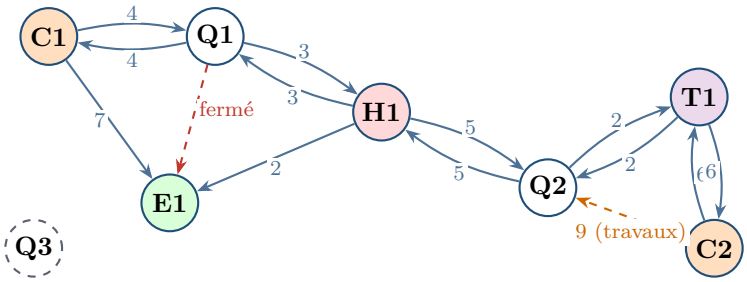


Figure 3 – Les routes du jeu `mini`. En orange, une route en travaux (temps multiplié par 1,5) ; en rouge, une route fermée, qui est donc retirée. Q3 n'a aucune route.

Voici ce qu'on a obtenu à la main, et que les tests vérifient maintenant automatiquement. Le BFS lancé depuis les deux casernes visite les sites dans l'ordre C1, C2, Q1, E1, T1, Q2 puis H1, qui est le plus loin (deux routes à prendre). Q3 n'est jamais atteint. Si on cherche le chemin de C2 à H1 qui prend le moins de routes, on trouve C2, Q2, H1 : deux routes, 14 minutes. Mais le chemin le plus rapide passe par T1 et prend trois routes pour seulement 13 minutes. C'est un bon exemple du fait que « le moins d'étapes » et « le plus rapide », ce n'est pas la même chose. Le DFS trouve aussi un circuit, C1 vers Q1 puis retour à C1 : il le détecte quand il tombe sur un site qui est encore en cours de visite. Enfin, on compte 3 composantes fortement connexes (le gros bloc de six sites, E1 tout seul et Q3 tout seul) mais seulement 2 composantes faiblement connexes, puisque E1 est quand même relié au reste si on oublie le sens des rues.

3.3 Résultats sur la ville complète

Un seul site ne peut pas être atteint : `Q-123`, qui n'a aucune route. On l'a mis exprès dans les données pour tester ce cas. Le circuit trouvé n'est pas très impressionnant, c'est juste une rue à double sens (on va de C-01 à Q-001 et on revient), mais ça montre que la détection marche. En plus des parcours, on a ajouté un seuil de 10 minutes, parce que pour la mairie « être atteignable » ne suffit pas, il faut aussi que les secours arrivent à temps. Huit quartiers, en plus de `Q-123`, dépassent ce seuil. Le pire est `Q-040`, à 11,9 minutes.

4 Module 2 — Itinéraires des secours

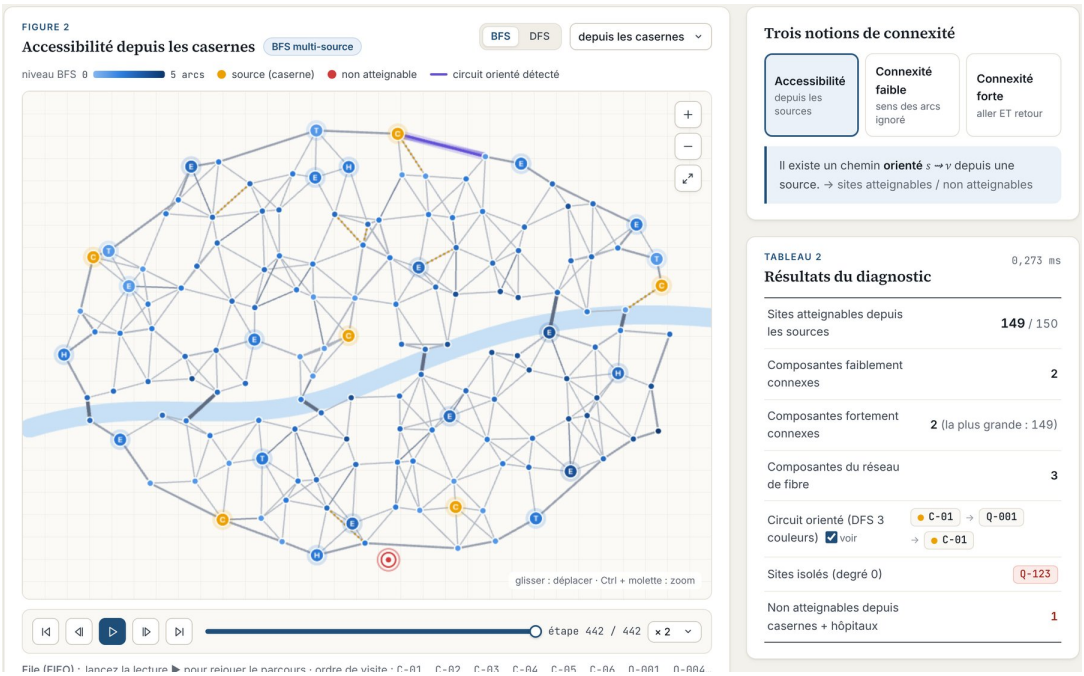


Figure 4 – Le module 1 dans l'interface. Les couleurs indiquent à combien de routes chaque site se trouve des casernes (plus c'est foncé, plus c'est loin). À droite, le diagnostic et le seuil de 10 minutes.

Indicateur	Notion / algorithme	Valeur
Sites atteignables depuis les 6 casernes	accessibilité, BFS	149 / 150
Composantes faiblement connexes	connexité faible	2
Composantes fortement connexes	connexité forte, Tarjan	2 (la plus grande : 149)
Composantes du réseau de fibre	connexité, BFS	3
Circuit orienté trouvé	DFS	C-01 → Q-001 → C-01
Site isolé	—	`Q-123`
Quartiers à plus de 10 min	Dijkstra multi-source	9 (dont `Q-123`, inatteignable)

Table 2 – Diagnostic d'accessibilité de la ville en situation normale (`novapolis access -data data/novapolis`).

4.1 Trois algorithmes qui doivent trouver la même chose

On a codé les trois algorithmes de plus court chemin vus en cours. **Dijkstra** est celui qu'on utilise par défaut, parce qu'il est le plus rapide quand tous les temps sont positifs, ce qui est le cas dans une vraie ville. Il refuse d'ailleurs de se lancer si le graphe contient un poids négatif, puisque dans ce cas il pourrait donner un résultat faux. **Bellman–Ford** est plus lent, mais il accepte les poids négatifs : il repasse sur toutes les routes, tour après tour, tant que ça améliore quelque chose. On l'arrête dès qu'un tour ne change plus rien, et on fait un tour de plus pour détecter les circuits négatifs (module 3). **Floyd–Warshall** calcule d'un coup les distances entre toutes les paires de sites, ce qui coûte beaucoup plus cher mais donne tout le tableau des distances.

Pour chaque algorithme, on garde de quoi reconstruire le chemin complet, pas seulement sa longueur. Pour chaque demande, l'outil affiche la durée, le nombre de routes empruntées, la caserne qui intervient, et les fermetures qui ont gêné le trajet. Sur le jeu `mini`, les trois algorithmes donnent exactement le même tableau de distances, et sur 50 graphes aléatoires ils trouvent les mêmes distances que `networkx` (section 8).

4.2 Et si on ferme les trois ponts du centre ?

Pour ce scénario, on ferme les trois ponts du centre qui traversent la Nova, dans les deux sens. On recalcule ensuite tous les itinéraires sur une copie de la ville et on compare avec la situation normale.

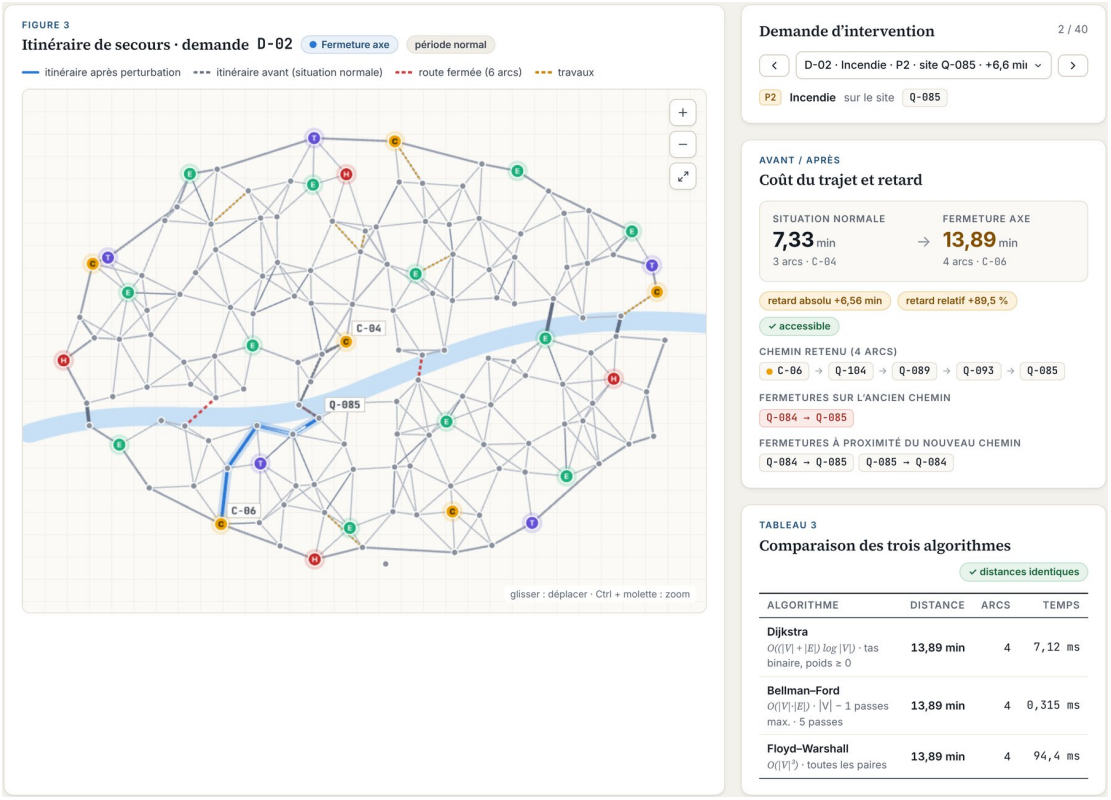


Figure 5 – La demande `D-02` (un incendie) avec les ponts fermés. Avant, les pompiers de `C-04` passaient par le pont entre `Q-084` et `Q-085`. Maintenant, c'est `C-06` qui intervient en restant du même côté de la rivière, mais le trajet passe de 7,33 à 13,89 minutes.

Demande	Type	Site	Avant	Après	Retard	En %	Caserne
`D-02`	Incendie (P2)	`Q-085`	7,33	13,89	+6,56	+89,5 %	C-04 → C-06
`D-03`	Fuite de gaz (P1)	`Q-080`	9,75	13,46	+3,71	+38,1 %	C-04 → C-05
`D-04`	Effondrement (P1)	`Q-093`	9,60	11,62	+2,02	+21,0 %	C-04 → C-06
`D-05`	Fuite de gaz (P2)	`Q-098`	10,52	11,49	+0,97	+9,2 %	C-04 → C-06
Les 35 autres demandes ne changent pas. `D-01` (sur `Q-123`) reste inaccessible dans les deux cas.							

Table 3 – Les demandes retardées quand les ponts du centre sont fermés (temps en minutes, `novapolis routes -scenario fermeture_axe`).

Pour `D-02`, les trois algorithmes trouvent le même trajet de 13,89 minutes, qui passe par `Q-104`, `Q-089` et `Q-093`. Par contre ils ne mettent pas le même temps à le trouver : 0,13 ms pour Dijkstra, 0,24 ms pour Bellman–Ford, et 49,5 ms pour Floyd–Warshall, qui calcule les distances entre toutes les paires de sites alors qu'on n'en utilise qu'une seule. Ce qui est intéressant, c'est que toutes les demandes retardées étaient servies par `C-04`, la caserne du centre. Sans les ponts, elle ne peut plus traverser, et il faut faire venir une caserne de l'autre rive. Si on regarde la moyenne, le retard paraît minuscule (0,34 minute sur les 39 demandes accessibles). Mais le pire cas est de 6,56 minutes, ce qui est énorme pour un incendie. C'est pour ça qu'on donne toujours la moyenne *et* le maximum.

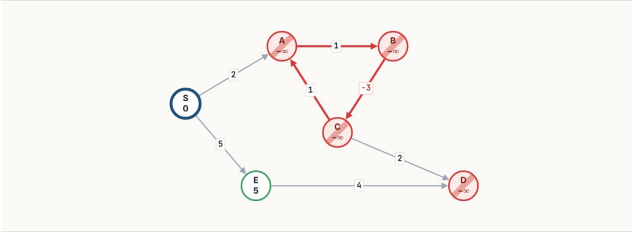
Les limites de notre modèle

Il faut garder en tête qu'un plus court chemin, ce n'est pas une vraie intervention. Nos temps sont fixes : il n'y a pas de trafic en direct, pas de feux rouges, et les véhicules de secours n'ont pas de priorité. On traite aussi une seule intervention à la fois, sans se demander si la caserne est déjà occupée : si `C-06` part pour `D-02`, en réalité elle ne pourrait pas aussi partir pour `D-04`. Le facteur 1,5 des travaux et les temps de pointe sont des hypothèses, pas des mesures. Le temps calculé ne compte pas non plus le temps de départ, l'accès au bâtiment ou le type de camion. Et comme les données sont inventées, les valeurs exactes n'ont pas de sens réel : c'est la méthode qui pourrait être réutilisée sur une vraie ville.

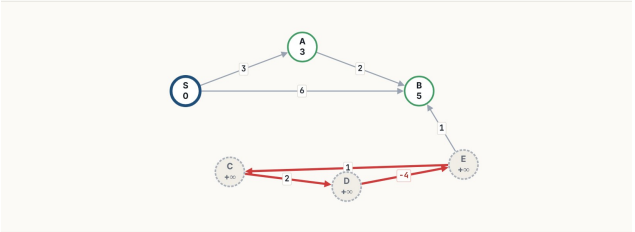
5 Module 3 — Les circuits absorbants

Un circuit absorbant, c'est une boucle dont le poids total est négatif. Si on tourne dedans, la distance baisse à chaque tour, sans jamais s'arrêter. Dans ce cas, il n'y a pas de « plus court chemin » : la distance vaut moins l'infini. Dans une vraie ville ça ne peut pas arriver, puisqu'un temps de trajet est toujours positif. On a donc créé deux petits graphes de test exprès, avec des poids négatifs clairement marqués comme artificiels.

Pour les détecter, on utilise les deux algorithmes qui acceptent les poids négatifs. Avec **Bellman–Ford**, après le nombre normal de tours, on en refait un de plus. Si une distance peut encore baisser, c'est qu'il y a un circuit absorbant atteignable depuis la source. Dans ce cas, on met $-\infty$ pour ce site et pour tous les sites qu'on peut atteindre à partir de lui. Avec **Floyd–Warshall**, c'est plus direct : si la distance d'un site vers lui-même devient négative, ce site est sur un circuit absorbant, et toutes les distances qui peuvent passer par lui valent $-\infty$.



(a) Test A : le circuit A, B, C (poids total -1) est atteignable depuis S.



(b) Test B : le circuit C, D, E (poids total -1) n'est pas atteignable depuis S.

Figure 6 – Les deux graphes de test du module 3 dans l'interface (`novapolis negcycle -algo all`).

	Test A (atteignable)		**Test B (non atteignable)**	
Site	Bellman–Ford	Floyd–Warshall	Bellman–Ford	Floyd–Warshall
S	0	0	0	0
A	$-\infty$	$-\infty$	3	3
B	$-\infty$	$-\infty$	5	5
C	$-\infty$	$-\infty$	$+\infty$	$+\infty$
D	$-\infty$	$-\infty$	$+\infty$	$+\infty$
E	5	5	$+\infty$	$+\infty$
Verdict	circuit trouvé, atteignable depuis S		circuit trouvé, pas atteignable	

Table 4 – Distances depuis S. Quand une distance peut baisser sans fin, l'outil affiche $-\infty$ et jamais un nombre.

Dans le test A, le site D vaut lui aussi $-\infty$, alors qu'il ne fait pas partie du circuit. C'est normal : on peut l'atteindre en sortant du circuit, donc sa distance peut baisser autant qu'on veut. E, lui, garde une distance normale (5), parce qu'aucun chemin ne va du circuit vers E. Le test B montre l'autre

situation. Le circuit existe bien, et les deux algorithmes le trouvent, mais depuis S on ne peut pas y aller, donc les distances depuis S restent normales ou infinies. C'est pour ça que l'outil donne deux informations séparées : « il y a un circuit absorbant » et « il est atteignable depuis la source ». Et si on prend C comme départ dans le test B, on retrouve bien $-\infty$ (ce cas est testé aussi).

6 Module 4 — Le réseau de fibre

6.1 Kruskal, Prim et Union–Find

Pour la fibre, on cherche à relier tous les sites en payant le moins possible, c'est-à-dire un arbre couvrant de poids minimum. **Kruskal** trie tous les tronçons du moins cher au plus cher, et les ajoute un par un, sauf ceux qui formeraient une boucle. Pour savoir rapidement si un tronçon crée une boucle, on utilise une structure **Union–Find**, qui garde en mémoire quels sites sont déjà reliés entre eux. On l'a codée avec les deux optimisations classiques (compression de chemin et union par rang), ce qui la rend presque instantanée. **Prim** fait l'inverse : il part d'un site et fait grandir l'arbre en ajoutant toujours le tronçon le moins cher qui touche l'arbre. Quand le graphe n'est pas connexe, il recommence depuis un site pas encore relié, ce qui donne une **forêt** couvrante (plusieurs arbres) au lieu d'un seul arbre.

Sur le petit jeu, on a fait le calcul à la main avant de comparer. Kruskal garde les tronçons Q2–T1, Q1–H1, E1–H1, T1–C2, C1–Q1 et H1–Q2, pour un total de 55 k€. Il refuse C2–Q2 parce que C2, T1 et Q2 sont déjà reliés. Prim, lui, garde C2–Q2 à la place de T1–C2. Les deux coûtent 9, et c'est juste C2–Q2 qui est arrivé en premier dans sa file. On a donc le même coût, mais pas le même réseau.

Au début, ça peut surprendre, mais c'est tout à fait normal. Quand on ajoute à l'arbre un tronçon qui n'en fait pas partie, ça crée une boucle. Si un autre tronçon de cette boucle coûte exactement le même prix, on peut échanger les deux et on obtient un autre arbre, aussi bon. Il n'y a qu'un seul arbre minimum quand tous les coûts sont différents, et ce n'est pas notre cas puisque nos coûts sont arrondis à 5 k€. L'outil cherche d'ailleurs automatiquement un exemple d'échange pour le montrer.

6.2 Résultats sur la ville

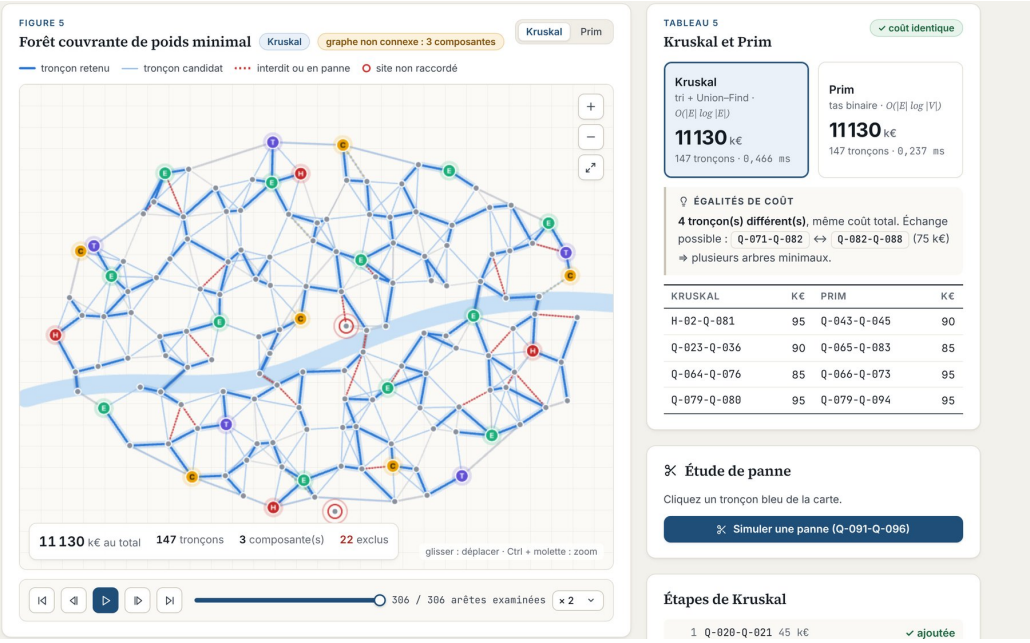


Figure 7 – Le réseau de fibre le moins cher (en bleu). Il y a 3 morceaux séparés, parce que `Q-058` et `Q-123` n'ont aucun tronçon disponible. Le tableau à droite compare Kruskal et Prim.

Comme sur le petit jeu, les deux algorithmes arrivent au même coût, avec 4 tronçons différents, à chaque fois à cause d'égalités de prix. Par exemple, on peut remplacer `Q-071`–`Q-082` par `Q-082`–`Q-088` : les deux coûtent 75 k€. Les deux sites qui restent hors réseau, `Q-058` et `Q-123`, sont signalés. Pour les

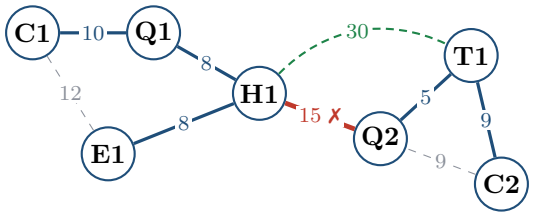
Algorithme	Coût total	Tronçons	Composantes	Temps
Kruskal	11 130 k€	147	3	0,23 ms
Prim	11 130 k€	147	3	0,13 ms

Table 5 – Kruskal et Prim sur la ville (`novapolis mst -algo both`). Avec 150 sites et 3 morceaux, une forêt couvrante a bien $150 - 3 = 147$ tronçons.

raccorder, la ville devrait autoriser un tronçon aujourd'hui interdit, ou en prévoir un nouveau.

6.3 Que faire en cas de panne ?

Si un tronçon de l'arbre tombe en panne, l'arbre se coupe en deux morceaux. Pour réparer, pas besoin de tout recalculer : il suffit de trouver le tronçon le moins cher qui relie ces deux morceaux, parmi ceux qui sont disponibles. Tout le reste du réseau est gardé.



Panne de H1–Q2 : il reste deux morceaux, {C1, Q1, H1, E1} et {Q2, T1, C2}
Le seul tronçon qui les relie est H1–T1 (30 k€), donc le nouveau coût est $55 - 15 + 30 = 70$ k€

Figure 8 – Exemple de panne sur le petit jeu. On l'a calculé à la main, et l'outil trouve la même chose (`novapolis mst-panne -data data/mini -edge H1-Q2`).

Sur la ville complète, on a testé la panne du tronçon `Q-014`–`Q-019` (65 k€), qui coupe le réseau en deux gros morceaux de 77 et 71 sites. Le meilleur raccordement est `Q-014`–`Q-025`, à 80 k€. Le coût total du réseau passe de 11 130 à 11 145 k€, soit seulement 15 k€ de plus, puisqu'on retire un tronçon à 65 et qu'on en ajoute un à 80. Mais en pratique, la ville doit quand même payer 80 k€ pour construire le nouveau tronçon. C'est ce coût-là qu'on reprend comme indicateur dans le module 5.

7 Module 5 — Résilience et scénarios

7.1 Les scénarios étudiés

Le sujet demandait au moins trois scénarios. On en a fait six, tous décrits dans le fichier `scenarios.csv`. Les trois premiers après la situation normale correspondent à ceux imposés par le sujet.

Scénario	Ce qui change
`normal`	rien, c'est la référence
`fermeture_axe`	les 3 ponts du centre sont fermés dans les deux sens (fermeture d'un axe important)
`panne_fibre`	le tronçon `Q-014`–`Q-019` tombe en panne (panne de fibre)
`heure_pointe`	toutes les routes passent au temps de pointe
`zone_travaux`	25 routes autour de la caserne centrale `C-04` sont en travaux (zone de travaux)
`crue_nova`	les sites au bord de la rivière `E-07` et `Q-052` sont coupés, toutes leurs routes sont fermées

Table 6 – Les six scénarios étudiés.

Un point important : on ne modifie jamais la ville de départ. Chaque scénario travaille sur une copie complète. Pour le prouver, on calcule une empreinte (SHA-256) des données de base avant et après chaque calcul, et elle ne change jamais (elle commence par `85bd6e3e`). Un test le vérifie aussi, fichier par fichier.

7.2 Les indicateurs

Le sujet proposait cinq indicateurs et en demandait au moins deux. On a calculé les cinq pour chaque scénario.

Indicateur	normal	ferm. axe	panne fibre	pointe	travaux	crue
Demandes devenues inaccessibles	0	0	0	0	0	**1**
Hausse moyenne du trajet (min)	0,00	0,34	0,00	**3,55**	0,31	0,00
Hausse maximale du trajet (min)	0,00	6,56	0,00	**8,26**	2,28	0,00
Composantes fibre (avant réparation)	3	3	**4**	3	3	3
Coût du raccordement (k€)	0	0	**80**	0	0	0
Sites sensibles accessibles	100 %	100 %	100 %	100 %	100 %	**96,3 %**

Table 7 – Les indicateurs pour chaque scénario (`novapolis scenarios -all`). En gras, la pire valeur de chaque ligne.

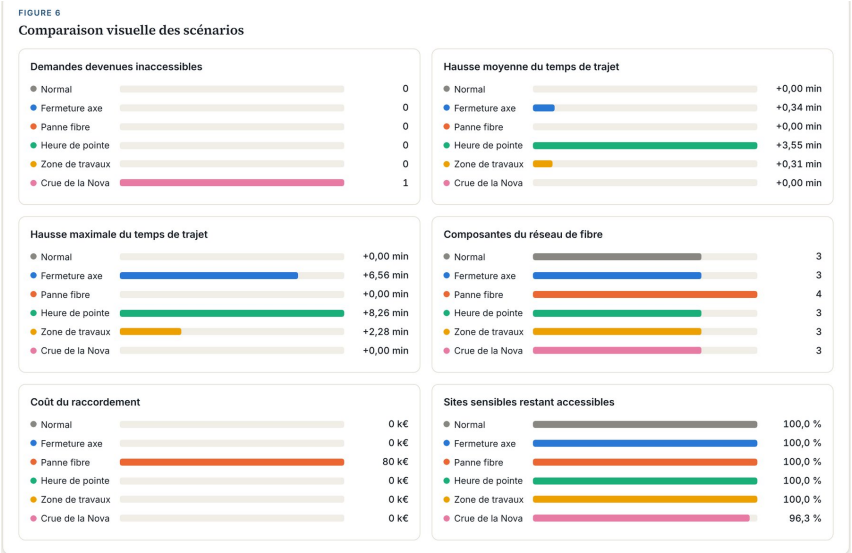


Figure 9 – Un petit graphique par indicateur, avec une barre par scénario. On voit que chaque scénario abîme quelque chose de différent.

7.3 Ce qu'on en retire

La première chose qu'on remarque, c'est que les scénarios ne touchent pas tous la même chose. La fermeture des ponts et l'heure de pointe rallongent les trajets, la panne abîme la fibre, et la crue rend des sites inaccessibles. Avec un seul indicateur, on serait passé à côté de deux scénarios sur trois.

L'heure de pointe est le scénario qui coûte le plus de temps au total. Elle touche presque tout le monde (39 demandes sur 40), avec 3,55 minutes de plus en moyenne et jusqu'à 8,26 minutes pour `D-27`. La fermeture des ponts, à l'inverse, ne touche que 4 demandes, mais beaucoup : c'est un risque très localisé. La zone de travaux reste locale elle aussi. Elle retarde 11 demandes, qui sont toutes servies par `C-04`, et d'au plus 2,28 minutes, ce qui correspond bien au facteur 1,5 sur des trajets courts.

La crue est un cas un peu particulier. Elle ne change aucun temps de trajet, mais l'incendie `D-29`, sur `Q-052`, devient impossible à atteindre, alors que c'est une priorité 1. L'école `E-07` est coupée aussi, et la part des sites sensibles accessibles descend à 96,3 %. La hausse moyenne reste à zéro, parce qu'on ne peut pas calculer de retard pour une demande qu'on n'atteint plus. Ça montre qu'il ne faut jamais lire cet indicateur seul, mais toujours avec le nombre de demandes devenues inaccessibles. Enfin, la panne de fibre coupe le réseau en 4 morceaux au lieu de 3. On peut la réparer pour 80 k€, et on revient à 3 morceaux.

8 Tests, robustesse et reproductibilité

On a écrit **151 tests** avec `pytest`. Ils passent tous, en environ 3 secondes (commande `pytest`). Le tableau ci-dessous résume ce qu'ils vérifient.

Fichier	Tests	Ce qui est vérifié
`test_mini_expected.py`	12	Tous les résultats calculés à la main sur le petit jeu : parcours, chemins, connexité, distances, travaux et pointe, scénarios, Kruskal et Prim, panne, indicateurs.
`test_cross_networkx.py`	100	Sur des graphes aléatoires, nos résultats sont les mêmes que ceux de `networkx` : 50 graphes pour les plus courts chemins, 20 pour la connexité, 30 pour l'arbre couvrant.
`test_edge_cases.py`	9	Les cas limites : graphe vide, un seul site, départ égal à l'arrivée, arrivée inaccessible, boucles, poids négatifs refusés par Dijkstra, toutes les routes fermées, chaîne de 20 000 sites.
`test_negative_cycles.py`	6	Les tests A et B, le test B vu depuis C, un graphe sans circuit, une boucle négative sur un seul site.
`test_data_pipeline.py`	17	Chaque erreur du validateur, les générateurs qui redonnent les mêmes fichiers, la ville de base jamais modifiée, et les mêmes chiffres dans le terminal, l'API et l'interface.
`test_lab.py`	7	Les animations pas à pas de l'interface arrivent bien au même résultat que l'algorithme.

Table 8 – Ce que vérifie notre suite de tests.

Pour la reproductibilité, toutes les données viennent d'une graine connue (42 pour la ville et pour les benchmarks), et les paramètres sont notés dans `meta.json`. La commande `novapolis data` régénère tous les jeux à l'identique. Chaque tableau de ce rapport donne la commande qui le produit, et dans l'interface, la page « Reproductibilité » fait la même chose pour chaque résultat affiché.

9 Mesures de performance et analyse

9.1 Comment on a mesuré

Pour que la comparaison soit juste, tous les algorithmes ont été lancés sur les mêmes graphes : 5 tailles (de 250 à 5 000 sites) et 3 densités (en moyenne 3, 6 ou 12 voisins par site), générés avec la graine 42. On chronomètre seulement l'algorithme, pas la génération ni le chargement des fichiers. Chaque mesure est refaite 5 fois, et on garde la moyenne et l'écart-type. Floyd–Warshall a été mesuré à part, sur des graphes plus petits (50 à 400 sites), parce qu'il devient vite beaucoup trop lent. Les commandes sont `novapolis bench -sizes 250,500,1000,2000,5000 -reps 5 -seed 42` puis `python scripts/make_figures.py`.

9.2 Résultats

Sur un graphique en échelles logarithmiques, un algorithme dont le temps grandit comme n^p donne une droite de pente p. Comme nos graphes ont toujours à peu près le même nombre de voisins par site, le nombre de routes grandit comme le nombre de sites. On peut donc comparer directement la pente mesurée avec celle que prévoit la complexité théorique (tableau 9).

9.3 Pourquoi les mesures s'écartent parfois de la théorie

Pour les parcours, Dijkstra, Kruskal et Prim, les mesures suivent bien la théorie, avec des pentes entre 1,0 et 1,15. Le petit facteur logarithmique ne se voit presque pas : entre 250 et 5 000 sites, le logarithme n'est multiplié que par 1,5. En Python, ce qui coûte le plus cher, c'est surtout d'exécuter chaque instruction, bien plus que de gérer le tas.

On voit aussi que le DFS est 2 à 4 fois plus lent que le BFS, alors qu'ils ont la même complexité. Notre

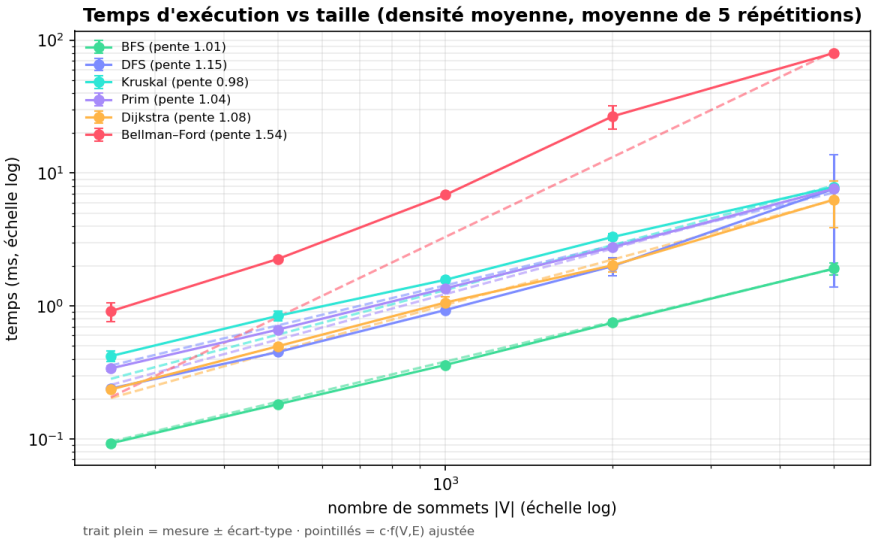
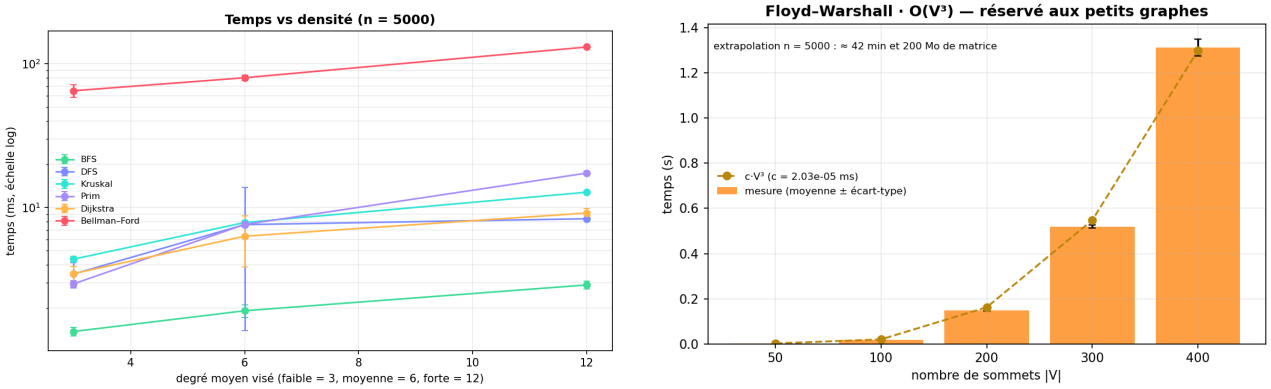


Figure 10 – Temps de calcul en fonction du nombre de sites, sur des échelles logarithmiques (densité moyenne). En trait plein, nos mesures ; en pointillés, la courbe que prévoit la théorie.



(a) Temps en fonction de la densité, à taille fixée.

(b) Floyd–Warshall, jusqu'à 400 sites.

Figure 11 – Effet de la densité, et coût de Floyd–Warshall.

DFS sans récursion doit gérer lui-même une pile un peu compliquée pour visiter les sites dans le même ordre que la version récursive. Ça ajoute du travail à chaque étape, et ce genre de différence ne se voit pas dans une notation en O.

Le résultat le plus intéressant concerne Bellman–Ford. La théorie donne une pente de 2, mais on mesure environ 1,5. L'explication vient de notre arrêt anticipé. On arrête l'algorithme dès qu'un tour ne change plus rien, et le nombre de tours nécessaires correspond en gros au nombre de routes du plus long des plus courts chemins, pas au nombre de sites. Le benchmark enregistre ce nombre de tours : 23 tours pour 250 sites et 96 pour 5 000 sites (densité faible), très loin des 4 999 du pire cas. Dans des graphes qui ressemblent à des villes, cette longueur grandit à peu près comme la racine carrée du nombre de sites, ce qui donne un coût en $n^{1,5}$: c'est exactement ce qu'on mesure. La complexité théorique reste juste, mais c'est un pire cas qu'on n'atteint pas ici.

Floyd–Warshall a une pente de 2,91, très proche du 3 attendu. Avec 400 sites, il met déjà 1,3 seconde. Si on extrapole au plus gros de nos jeux (5 000 sites), il faudrait environ 43 minutes, et une matrice de 25 millions de cases qui prendrait plusieurs centaines de Mo en Python. C'est pour ça qu'on le garde pour les petits graphes et les comparaisons. Pour un seul départ, Dijkstra va beaucoup plus vite : sur la ville, 0,13 ms contre 49,5 ms, soit plus de 300 fois plus rapide.

Quand on augmente la densité à taille fixée, le temps augmente à peu près proportionnellement au

	Pente mesurée				Temps moyen à 5 000 sites	
Algorithme	faible	moyenne	forte	**Théorie**	moyenne	forte
BFS	1,06	1,01	1,00	1	1,91 ms	2,89 ms
DFS	1,07	1,15	1,00	1	7,60 ms	8,35 ms
Kruskal	1,00	0,98	1,06	environ 1,1	7,86 ms	12,8 ms
Prim	0,99	1,04	1,15	environ 1,1	7,62 ms	17,4 ms
Dijkstra	1,04	1,08	1,05	environ 1,1	6,32 ms	9,16 ms
Bellman–Ford	1,46	1,54	1,41	2	80,0 ms	131 ms
Floyd–Warshall	—	2,91	—	3	1 312 ms à 400 sites	

Table 9 – Pentes mesurées comparées à la théorie. Le facteur logarithmique de certains algorithmes ajoute à peu près 0,1 à la pente sur cette plage de tailles.

nombre de routes pour tous les algorithmes (figure 11a), ce qui est cohérent avec leurs complexités. Enfin, quelques mesures ont un écart-type assez grand, comme Prim à 2 000 sites en densité forte ou le DFS à 5 000 sites. On pense que ça vient du ramasse-miettes de Python, qui se déclenche parfois pendant une mesure. Avec plus de répétitions et la médiane au lieu de la moyenne, les résultats seraient plus stables.

10 Limites et recommandations

Notre travail a évidemment des limites. Les données sont inventées : la méthode fonctionne, mais les chiffres ne décrivent pas une vraie ville. Pour l'appliquer pour de vrai, il faudrait utiliser des données ouvertes, comme OpenStreetMap, en citant la source et la licence. Le modèle est aussi statique et traite une intervention à la fois, sans trafic en direct, sans gérer les casernes déjà occupées ou les incidents simultanés. Les coûts de fibre sont arrondis et supposés, et le raccordement le moins cher ne tient pas compte des délais de travaux ou du risque d'une nouvelle panne au même endroit. Enfin, on a testé quelques scénarios choisis, pas toutes les pannes possibles. La suite logique serait de retirer chaque pont et chaque tronçon un par un, pour trouver automatiquement les points les plus fragiles.

À partir de nos résultats, voici ce qu'on recommanderait à la ville :

1. **Désenclaver `Q-123` et raccorder `Q-058` à la fibre.** Ce sont les deux seuls sites hors réseau, et à cause de `Q-123`, une demande reste inaccessible quel que soit le scénario.
2. **Sécuriser les ponts du centre.** Leur fermeture retarde 4 interventions, jusqu'à presque deux fois plus de temps. Un itinéraire de délestage, ou une petite caserne relais sur la rive sud près de `Q-085`, réduirait beaucoup ce risque.
3. **Doubler la fibre là où une panne coupe le réseau en deux gros morceaux**, comme pour `Q-014`–`Q-019` (77 et 71 sites). Préparer à l'avance `Q-014`–`Q-025` (80 k€) permettrait de réparer tout de suite.
4. **Surveiller les 8 quartiers entre 10 et 12 minutes**, comme `Q-040` (11,9 minutes). Ce sont les premiers qui sortiront du délai à l'heure de pointe.
5. **Protéger `Q-052` et `E-07` contre les crues**, puisque c'est le seul scénario où l'on perd une intervention de priorité 1.

11 Démonstration et contributions

Pour la démonstration de 12 minutes, l'interface a un mode « Présenter ». C'est une visite guidée en 12 étapes qui suit l'ordre du sujet : la vue d'ensemble, le validateur, le BFS rejoué étape par étape, la demande `D-02` avant et après la fermeture des ponts, les tests A et B, la comparaison de Kruskal et Prim avec la panne, le tableau des scénarios, les benchmarks, et enfin la grille de conformité. Si le

jury pose une question précise, la page « Algorithmes en direct » permet de relancer n'importe quel algorithme sur le graphe et le départ de son choix.

Membre	Modules principaux	Fichiers et livrables
Hassan AJDAHIM (membre 1)	M2 Itinéraires, M5 Scénarios, interface	`shortest_paths.py`, `indicators.py`, `scenario.py`, `ui/` ; démonstration
Andreja JOVANOVIC (membre 2)	M4 Fibre, Union–Find, benchmarks	`mst.py`, `union_find.py`, `bench.py`, figures
Angelo CUNHA VELHO MACHADO (membre 3)	M1 Parcours, M3 Circuits absorbants, données	`traversal.py`, `negative_cycles.py`, `io/`, `generator.py`
Travail commun : modélisation, format des données, petit jeu calculé à la main, rapport, répétition de la démonstration.		

Table 10 – Qui a fait quoi (le détail est dans `CONTRIBUTIONS.md`). Chacun de nous est capable d'expliquer tous les algorithmes.

Document joint. En plus de ce rapport, nous joignons un fichier séparé, `correspondance_sujet_code.pdf` (7 pages, aussi disponible dans le dépôt, dossier `reports/`). Pour chaque exigence du sujet, il indique le fichier et la fonction qui la réalisent, comment elle est réalisée, et la commande ou le test qui permet de la vérifier. Il n'est pas inclus ici pour ne pas dépasser la limite de pages, mais il peut servir de guide pour retrouver rapidement chaque point dans le code.

A Annexe — Commandes pour tout refaire

Résultat	Commande
Installation	`python -m venv .venv; pip install -e ".[dev]"; npm -prefix ui install`
Interface complète	`python tasks.py demo` (puis `http://127.0.0.1:8000`)
Régénérer les données	`novapolis data`
Validation	`novapolis validate -data data/novapolis`
M1 (fig. 4)	`novapolis access -data data/novapolis -sources casernes`
M1, chemin le plus court en routes	`novapolis minhop -data data/mini -from C2 -to H1`
M2 (tab. 3)	`novapolis routes -data data/novapolis -scenario fermeture_axe`
M2, demande D-02 (fig. 5)	`novapolis route -demande D-02 -algo all -scenario fermeture_axe`
M3 (tab. 4)	`novapolis negcycle -data data/negatif_accessible -source S -algo all`
M4	`novapolis mst -data data/novapolis -algo both`
M4, panne	`novapolis mst-panne -data data/novapolis -edge Q-014-Q-019`
M5 (tab. 7)	`novapolis scenarios -data data/novapolis -all`
Benchmarks	`novapolis bench -sizes 250,500,1000,2000,5000 -reps 5 -seed 42`
Figures	`python scripts/make_figures.py reports/bench.csv`
Tests	`pytest`